

■ ■ Diagrama C4 — CliniDesk

Contexto, Containers e Componentes do Sistema NPI

Projeto: CliniDesk — NPI

Versão: Sprint 2 · 2026

Ano: 2026

Modelo C4 — Visão Arquitetural

O modelo C4 (Context, Containers, Components, Code) é uma abordagem hierárquica para documentar a arquitetura de software. Este documento cobre os níveis N1 (Contexto) e N2 (Containers) do sistema CliniDesk, com descrição funcional dos módulos do N3 (Componentes).

N1 — Contexto do Sistema

O contexto define **quem usa** o sistema e **com o que ele se conecta** externamente. O CliniDesk atua como o núcleo digital da recepção do NPI.

Atores do Sistema

Ator	Tipo	Interação com o Sistema
■■■■ Recepcionista	Usuário Primário	Agenda consultas, busca pacientes, envia lembretes e confirma presença
■ Gestor	Usuário Secundário	Confere histórico de mudanças e valida integridade dos dados
■ Idoso / Responsável	Receptor Externo	Recebe mensagem no WhatsApp e lembrete impresso — não acessa o sistema

Sistemas Externos

Sistema	Tecnologia	Integração
■ WhatsApp Web	Browser redirect + URL	Sistema gera mensagem formatada e abre WhatsApp Web com o chat pronto
■ Impressora	API de impressão do navegador	Geração de Fichete de consulta para impressão física ou exportação em PDF
■ MongoDB Atlas	Driver Mongoose via TCP/IP	Permanência de todos os dados na nuvem com autenticação segura

Diagrama de Contexto (N1)

```
[Recepcionista] ■■■■■■■■■■ [CliniDesk – Sistema Web]
[Gestor] ■■■■■■■■■■ ■
■■■■ [MongoDB Atlas]
■■■■ [WhatsApp Web]
■■■■ [Impressora]
[Idoso/Responsável] ■■■ mensagem ■■■
```

N2 — Containers

O nível de Containers mostra como o sistema é dividido em processos/serviços que se comunicam.

■■ Frontend (React + Vite)

[SPA — Single Page Application]

■ Porta 5173 · Pasta /Front

Responsabilidades:

- Exibe o Dashboard com tabela de agendamentos e alertas visuais (Smart Color-Coding)
- Abre modal de criação/edição de Consultas e Retornos
- Dispara requisições Axios para a API do backend (porta 5000)
- Gerencia autenticação de forma reativa — interceptor global captura 401 e redireciona ao login
- Gera link WhatsApp Web com mensagem pré-preenchida
- Aciona impressão do comprovante via API do navegador ou PDF

■■ Stack: React.js v18 · Vite · Axios · date-fns · lucide-react · Tailwind CSS

■■ Backend (Node.js + Express)

[API REST]

■ Porta 5000 · Pasta /backend · Arquivo: src/server.js

Responsabilidades:

- Valida token JWT via cookie HttpOnly em todas as rotas protegidas (middleware de autenticação)
- Aplica rate-limiting no endpoint de login (5 tentativas / 15 min) contra brute-force
- Valida dados de entrada via middleware validateAgendamento antes de persistir
- Executa CRUD de Consultas e Retornos: criação, listagem, atualização, cancelamento
- Registra lembrete: grava ID do usuário e timestamp do envio
- Gerencia sessão via cookie seguro (HttpOnly + SameSite + Secure)

■■ Stack: Node.js · Express · Mongoose · jsonwebtoken · bcryptjs · cookie-parser · express-rate-limit · dotenv

■■ Banco de Dados (MongoDB Atlas)

[NoSQL Document Store]

■ Hospedado na nuvem · Acesso via MONGO_URI no .env

Responsabilidades:

- Armazena documentos de Agendamentos (paciente, responsável, data, horário, setor, status)
- Armazena usuários (username, hash da senha, nível de acesso)
- Schema flexível permite adicionar campos sem migrations

- Conectado via Mongoose com suporte a validações de schema

■ ■ Stack: MongoDB Atlas · Mongoose ODM

N3 — Componentes Principais

Frontend — Componentes React

Componente	Responsabilidade
Dashboard.jsx	Painel principal: tabela de agendamentos, filtros reativos por data/setor, Smart Color-Coding
ModalNovoAgendamento.jsx	Formulário de criação de Consulta e Retorno, com validações via Guard Clauses e seleção
App.jsx	Roteamento SPA, gerenciamento de estado global de autenticação e interceptor Axios para
Login.jsx	Tela de autenticação com username e senha, chama /api/auth/login

Backend — Módulos Node.js

Módulo/Arquivo	Responsabilidade
routes/pacienteRoutes.js	Define rotas REST: GET/POST/PUT/DELETE de consultas e retornos
controllers/patientController.js	Lógica de negócio: processa requisições, aplica regras e persiste via Mongoose
middlewares/authMiddleware.js	Verifica e valida o cookie HttpOnly com o token JWT em rotas protegidas
middlewares/validateAgendamento.js	Valida telefone, data, horário, setor e campos obrigatórios antes de persistir
utils/validators.js	Funções puras de validação (testadas com Vitest)
utils/formatters.js	Funções puras de formatação e normalização de dados
utils/calculators.js	Regras de negócio: dias para consulta, disponibilidade de horário
models/Agendamento.js	Schema Mongoose do agendamento com validações de campo
models/User.js	Schema Mongoose do usuário com hash de senha via bcryptjs

Fluxo de Comunicação entre Containers

Browser (React)  Express API (Node.js)  MongoDB Atlas

▲ ■

■ validates JWT cookie

■■■■■ 401 Unauthorized ■■■■■■ (redireciona ao login)

■

■■■■■ WhatsApp Web (redirect URL)

■■■■■ Impressora (window.print / PDF blob)